# L10A: Searching and Sorting II; Memoization

CS1101S: Programming Methodology

Martin Henz

21st October, 2026

Version 1.0 — Built: 2026-10-20 23:19:38 SGT

# Outline

Searching

Sorting

Memoization

Searching

Sorting

Memoization

# Linear Search

- Inspect the list from left to right until the value is found.
- Accessing a list element by index takes $\Theta(1)$ time.
- The list may be in any order; it need not be sorted.

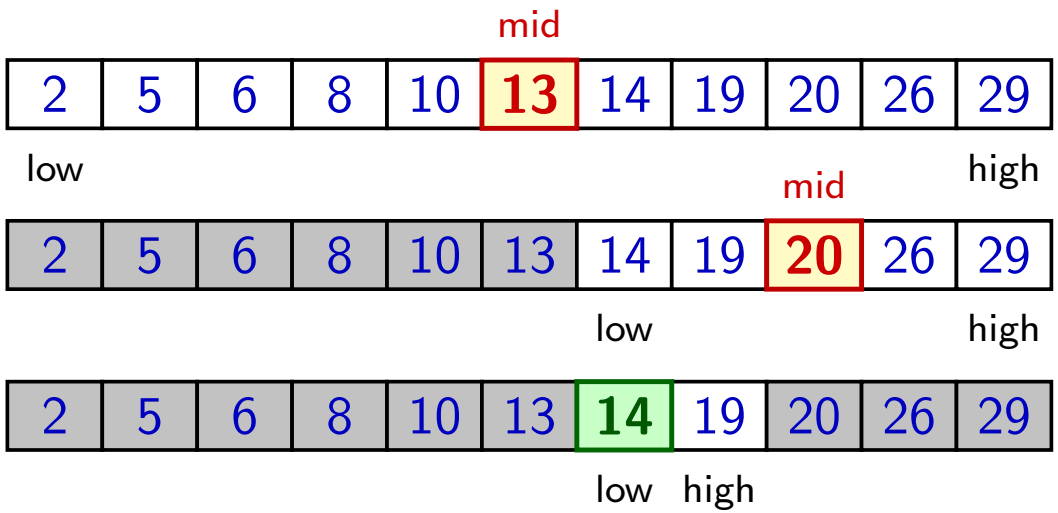


```
def linear_search(A, value):
    i = 0
    while i < len(A) and A[i] != value:
        i = i + 1
    return i < len(A)

print(linear_search([8, 2, 13, 5, 10, 6], 10))
```

# Binary Search

- If the list is sorted in ascending order, we can do better than linear search.
- Compare the target with the middle element.
- Discard the half in which the target cannot occur.




**target = 14**

$mid = \lfloor (0 + 10)/2 \rfloor = 5$

$A[5] = 13 < 14 \Rightarrow$ right

$mid = \lfloor (6 + 10)/2 \rfloor = 8$

$A[8] = 20 > 14 \Rightarrow$ left

$mid = \lfloor (6 + 7)/2 \rfloor = 6$

$A[6] = 14 \Rightarrow$ **found**

# Why Binary Search Is Fast

- Each comparison halves the remaining search interval.
- The number of comparisons is $\Theta(\log n)$.
- This relies on $\Theta(1)$ indexed access.
- A linked list therefore cannot support this binary-search algorithm efficiently.
- We needed a new data structure: binary search trees.

# Binary Search Using Recursion



```
def binary_search(A, value):
    def search(low, high):
        if low > high:
            return False
        mid = math_floor((low + high) / 2)
        if value == A[mid]:
            return True
        if value < A[mid]:
            return search(low, mid - 1)
        return search(mid + 1, high)
    return search(0, len(A) - 1)

print(binary_search([2, 5, 6, 8, 10, 13], 8))
```

# Binary Search Using a Loop



```
def binary_search(A, value):
    low = 0
    high = len(A) - 1
    while low <= high:
        mid = math_floor((low + high) / 2)
        if value == A[mid]:
            return True
        if value < A[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return False

print(binary_search([2, 5, 6, 8, 10, 13], 8))
```

Searching

Sorting

Memoization

# Swapping Values Does Not Mutate the List

```
def swap(x, y):
    temp = x
    x = y
    y = temp

A = [4, 5, 6, 7]
swap(A[0], A[3])
print(A)
```



- Reassigning `x` and `y` changes only their local bindings.
- The elements of `A` are unchanged.

# Swap the List Elements Directly

```
def swap(A, i, j):
    temp = A[i]
    A[i] = A[j]
    A[j] = temp
```



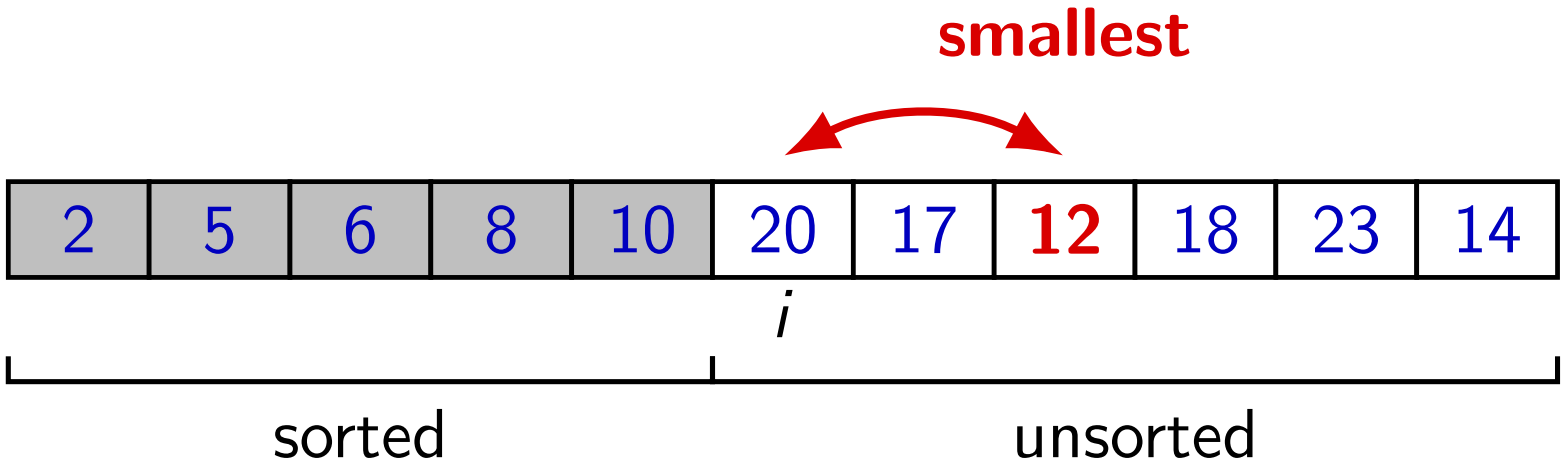
```
A = [4, 5, 6, 7]
swap(A, 0, 3)
print(A)
```



- Assignment to `A[i]` mutates the list itself.

# Selection Sort

- Grow a sorted region from left to right.
- Find the smallest element in the remaining unsorted region.
- Swap it into position $i$.



# Selection Sort for Lists

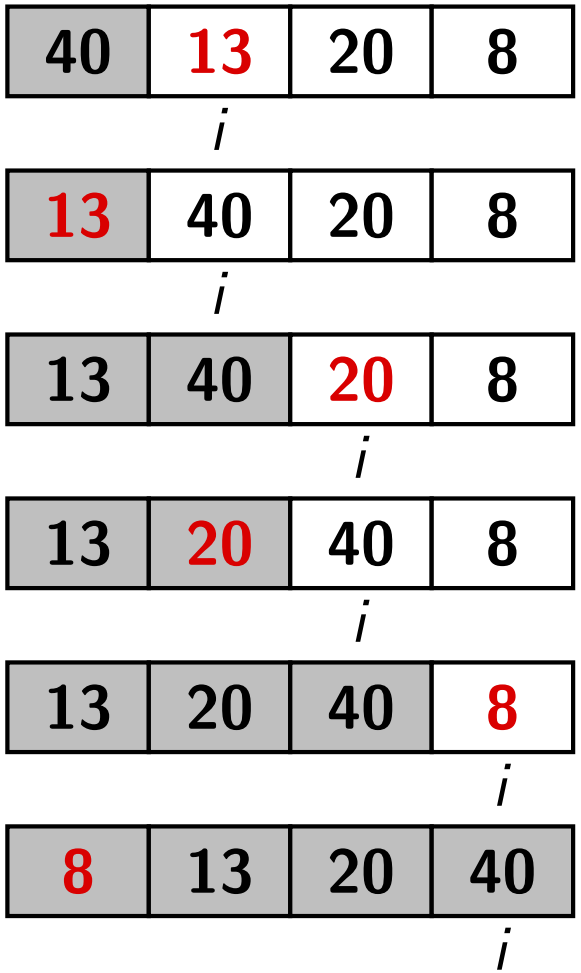


```
def selection_sort(A):
    for i in range(len(A) - 1):
        min_pos = i
        for j in range(i + 1, len(A)):
            if A[j] < A[min_pos]:
                min_pos = j
        swap(A, i, min_pos)

A = [10, 5, 8, 2, 6]
selection_sort(A)
print(A)
```

# Insertion Sort

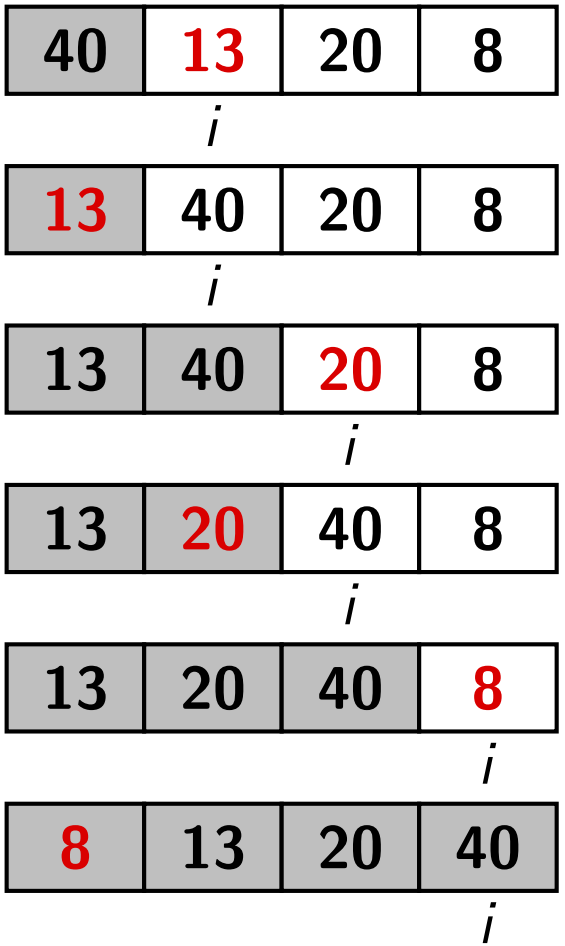
- The region to the left of $i$ is already sorted.
- Move the value at $i$ left until it reaches its correct position.



# Insertion Sort by Swapping

```
def insertion_sort(A):
    for i in range(1, len(A)):
        j = i
        while (j > 0 and
               A[j - 1] > A[j]):
            swap(A, j - 1, j)
            j = j - 1

A = [40, 13, 20, 8]
insertion_sort(A)
print(A)
```



# Insertion Sort by Shifting

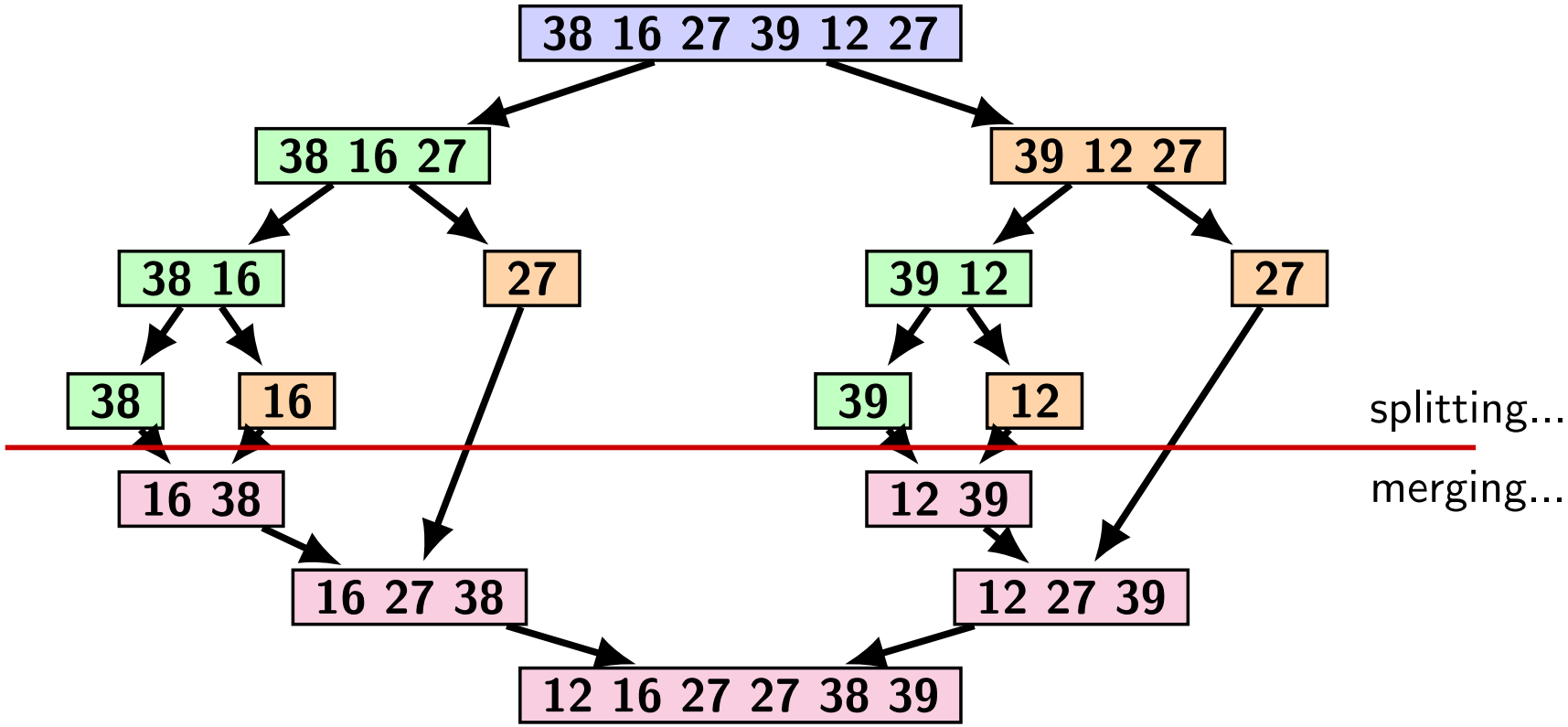
```
def insertion_sort(A):
    for i in range(1, len(A)):
        value = A[i]
        j = i - 1
        while j >= 0 and A[j] > value:
            A[j + 1] = A[j]
            j = j - 1
        A[j + 1] = value

A = [40, 13, 20, 8]
insertion_sort(A)
print(A)
```



# Merge Sort

- Split the list into halves until each part has one element.
- Merge adjacent sorted parts back together.



# Merge Sort for Lists

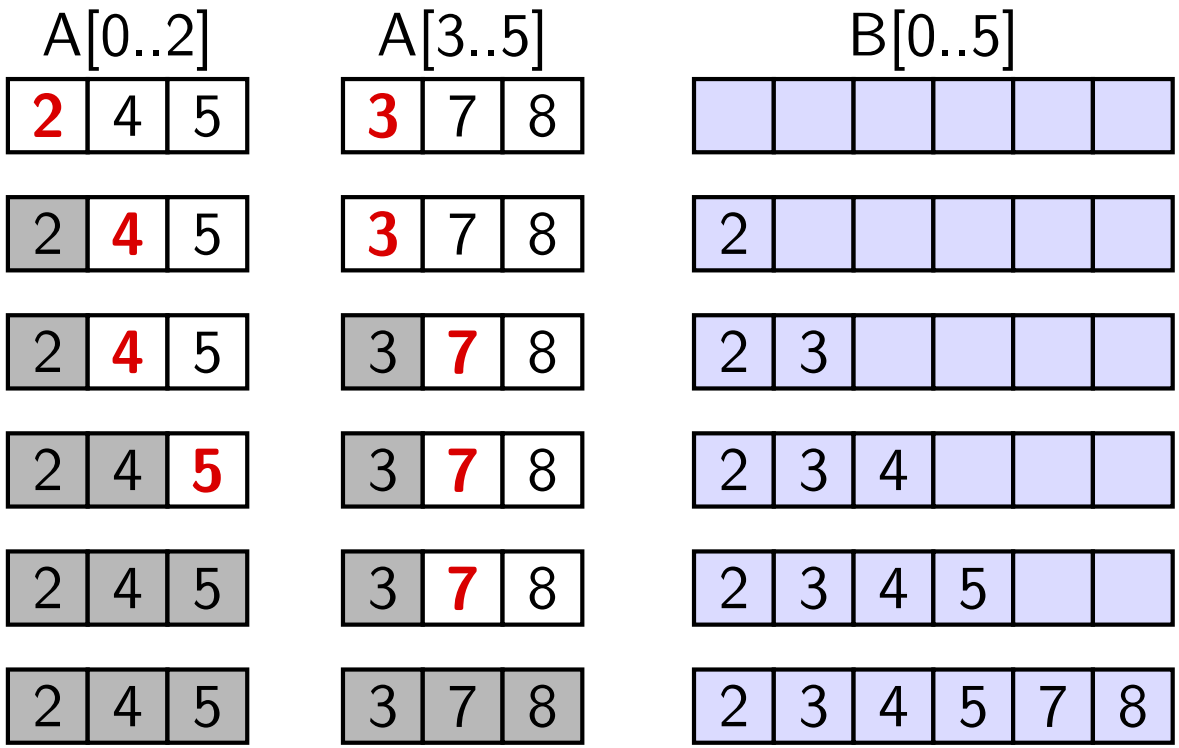
```
def merge_sort(A):
    def sort(low, high):
        if low < high:
            mid = math_floor((low + high) / 2)
            sort(low, mid)
            sort(mid + 1, high)
            merge(A, low, mid, high)
    sort(0, len(A) - 1)

A = [5, 2, 8, 3, 7, 4]
merge_sort(A)
print(A)
```



# Merging Two Sorted Regions

- Repeatedly copy the smaller front element into a temporary buffer.
- Copy the completed buffer back into the original list.



# The Merge Operation



```
def merge(A, low, mid, high):
    B = [None] * (high - low + 1)
    left = low
    right = mid + 1
    i = 0
    while left <= mid or right <= high:
        take_left = right > high or (
            left <= mid and A[left] <= A[right])
        if take_left:
            B[i] = A[left]
            left = left + 1
        else:
            B[i] = A[right]
            right = right + 1
        i = i + 1
    for i in range(len(B)):
        A[low + i] = B[i]
```

Searching

Sorting

Memoization

# Naive Fibonacci Repeats Work

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)

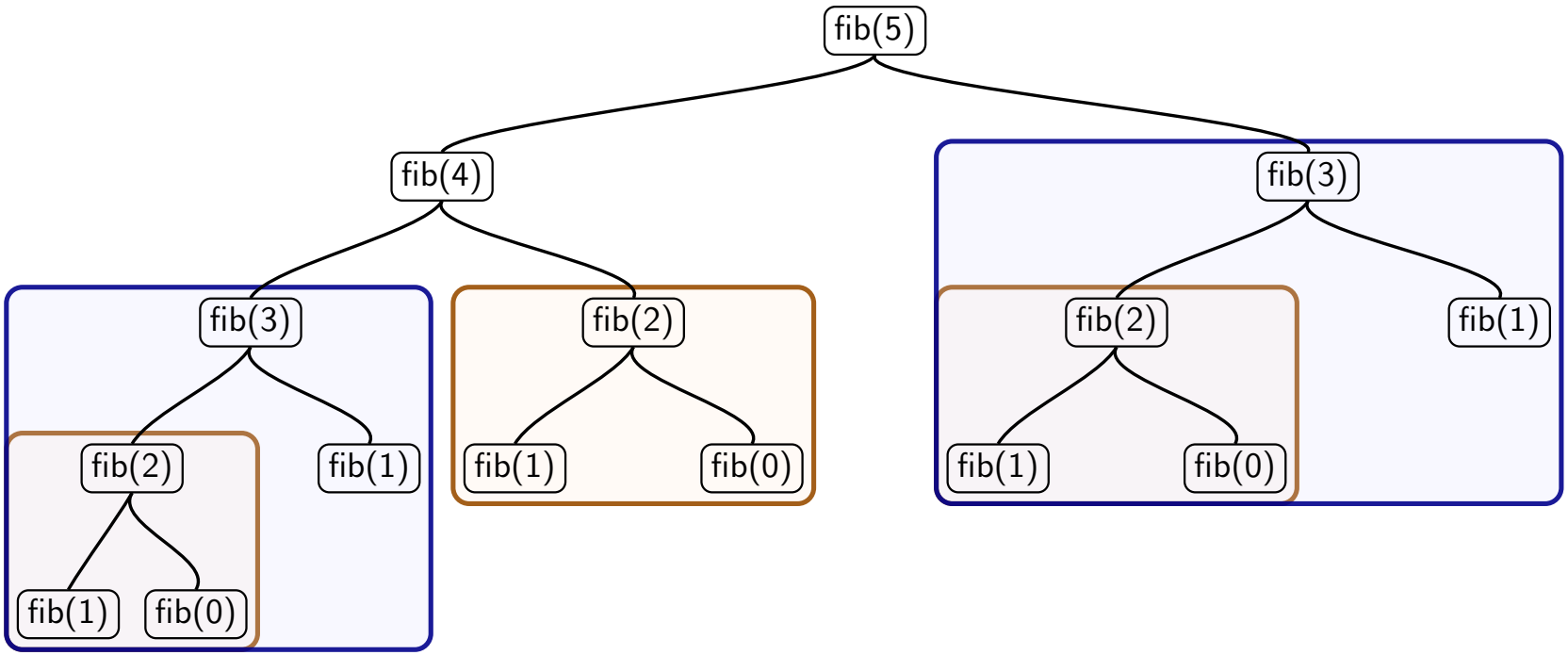
print(fib(5))
```



- Time: $\Theta(\varphi^n)$, where $\varphi$ is the golden ratio.

# Why Exponential?

- The same function calls are evaluated repeatedly.
- How can we avoid this repeated work?



# Memoization

- Record a result the first time it is computed.
- On later calls with the same argument, return the recorded result.
- For nonnegative integer arguments, a list gives $\Theta(1)$ lookup.

# Memoized Fibonacci with Local Memory

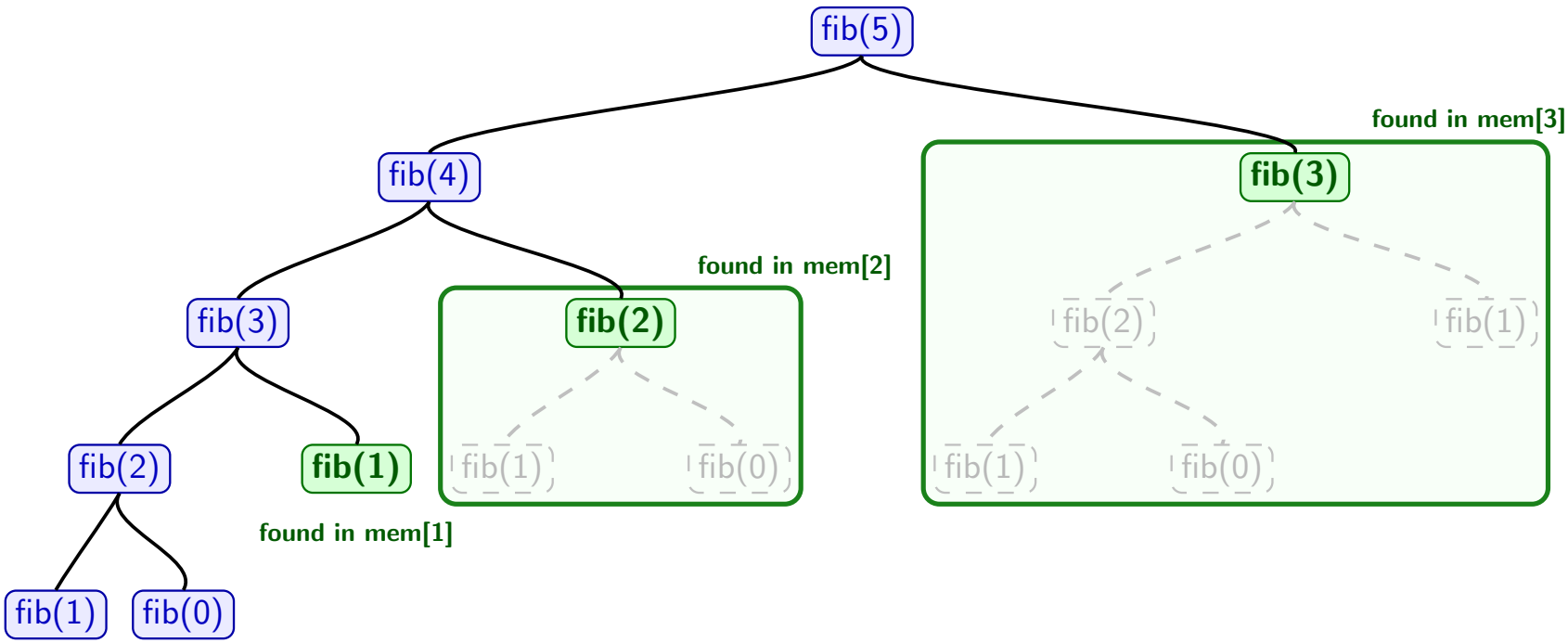


```
def mfib(n):
    mem = [None] * (n + 1)
    def fib(k):
        if mem[k] is not None:
            return mem[k]
        result = k if k <= 1 else fib(k - 1) + fib(k - 2)
        mem[k] = result
        return result
    return fib(n)

print(mfib(10))
```

# Evaluating `mfib(5)`

- Each distinct argument is fully evaluated only once.




- Time: $\Theta(n)$; additional space: $\Theta(n)$.

# An Abstraction for Memoization

```
def memoize(f, size):
    mem = [None] * size
    def memoized(x):
        if mem[x] is not None:
            return mem[x]
        result = f(x)
        mem[x] = result
        return result
    return memoized
```



- The returned function retains its private memory between calls.

# First Attempt: Memoize Only the Outer Call

```
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)

memo_fib = memoize(fib, 24)
print(memo_fib(23))
```



- This is still exponential: recursive calls invoke `fib`, bypassing `memo_fib`.

# Second Attempt: Memoized Recursion

```
mfib = memoize(
    lambda n: n if n <= 1
              else mfib(n - 1) + mfib(n - 2),
    32)

print(mfib(31))
```



- Every recursive call now passes through the memoized function.
- Time and additional space are both $\Theta(n)$.

# The $n$-Choose-$k$ Problem

- How many ways can we choose $k$ items from $n$ items?
- Pick one item:
  - Do not choose it: choose $k$ from the remaining $n-1$.
  - Choose it: choose $k-1$ from the remaining $n-1$.
- Therefore,

$$C(n,k) = C(n-1,k) + C(n-1,k-1).$$

# A Straightforward Implementation

```
def choose(n, k):
    if k > n:
        return 0
    if k == 0 or k == n:
        return 1
    return choose(n - 1, k) + choose(n - 1, k - 1)

print(choose(5, 2))
```



- Many pairs $(n, k)$ are evaluated repeatedly.

# Memoizing Two Arguments

- The result depends on the pair $(n, k)$.
- For one call `choose(n, k)`, all relevant arguments satisfy $0 \leq i \leq n$ and $0 \leq j \leq k$.
- A preallocated $(n + 1) \times (k + 1)$ table therefore gives $\Theta(1)$ lookup and update.

# Creating a Two-Dimensional Table

```
def make_table(rows, columns):
    table = [None] * rows
    for i in range(rows):
        table[i] = [None] * columns
    return table
```



- Each row is a distinct list, so updating one row does not change another.

# Memoized $n$-Choose-$k$



```
def mchoose(n, k):
    mem = make_table(n + 1, k + 1)
    def choose(i, j):
        if j > i:
            return 0
        if mem[i][j] is not None:
            return mem[i][j]
        result = 1 if j == 0 or j == i else (
            choose(i - 1, j) + choose(i - 1, j - 1))
        mem[i][j] = result
        return result
    return choose(n, k)

print(mchoose(5, 2))
```

# Memoized $n$-Choose-$k$: Cost

- Each reachable pair $(i, j)$ is computed at most once.
- Each computation performs $\Theta(1)$ work besides its recursive calls.
- Time: $\Theta(nk)$.
- Additional space: $\Theta(nk)$ for the table, plus recursion space.

# Summary

- Binary search exploits sorted lists and random access to achieve $\Theta(\log n)$ time.
- Selection, insertion, and merge sort adapt familiar list-processing ideas to mutable lists.
- Memoization replaces repeated evaluation with table lookup.
- The table representation determines the cost of that lookup.

# Check-In time!

<insert bad pun>

*Look for Check-In I10A in the Source Academy!*